

titanic dataset

# 쿠글 W1 세미나

14기 송채영

→ 데이터를 이해하고  
가공하는 전 과정

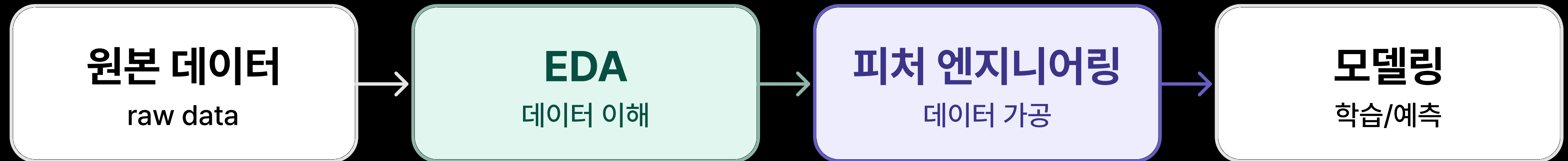
2026. 09. 08

E 피  
D 처  
A 엔  
& 지  
니  
어  
링



## Agenda

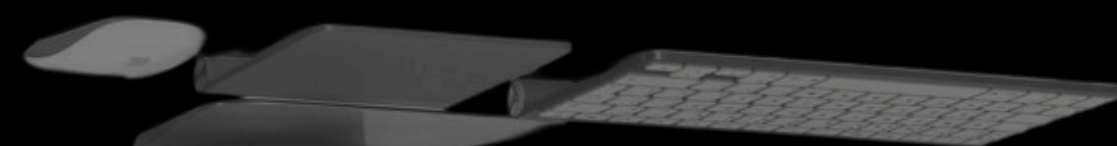
# 탐색적 자료분석(EDA)과 피처엔지니어링



# 01

titanic dataset

EDA란?





---

# 탐색적 자료분석 EDA :

---

데이터를 본격적으로 분석하기 전,  
( 그래프나 통계로 자료의 특징과 숨겨진 패턴 ) 을 미리 살펴보는 과정.

---

크게 4가지로 구성되어 있어요.

- 01 데이터 구조 파악 : 행과 열의 개수, 변수 타입 확인
  - 02 기초 통계량 확인 : 평균, 중앙값, 최빈값, 표준편차 계산
  - 03 결측값 및 이상치 탐지 : 비어 있는 값이나 유독 튀는 데이터 찾기
  - 04 데이터 시각화 : 히스토그램, 산점도, 상자 그림 등으로 분포와 관계 확인
-

EDA

# 탐색적 자료분석

## EDA LIBRARY

01

**Pandas**

데이터 조작과 분석

02

**Matplotlib**

데이터 시각화

03

**Seaborn**

Matplotlib 기반의  
고급 시각화

04

**Numpy**

선형 대수 함수 제공

파이썬에 내장된  
데이터셋을 활용해요

---

**Titanic**



EDA

# 데이터 구조 파악

- 891행 \* 15열 → 승객 1명이 한 행
- int64 / float64 : **수치형** (생존여부, 등급, 나이, 요금 등)
- str / category : **범주형** (성별, 항구, 등급명 등)
- bool : **참 / 거짓** (성인남성 여부, 혼자탑승 여부)
- Non-Null Count가 891보다 작은 컬럼 → **결측치 존재**

```
# 데이터 구조 파악
print(df.shape)
print(df.info())
```

(891, 15)

| #   | Column     | Non-Null Count | Dtype    |
|-----|------------|----------------|----------|
| 0   | survived   | 891 non-null   | int64    |
| 1   | pclass     | 891 non-null   | int64    |
| 2   | sex        | 891 non-null   | str      |
| 3   | age        | 714 non-null   | float64  |
| 4   | sibsp      | 891 non-null   | int64    |
| 5   | parch      | 891 non-null   | int64    |
| 6   | fare       | 891 non-null   | float64  |
| 7   | embarked   | 889 non-null   | str      |
| 8   | class      | 891 non-null   | category |
| ... | (7개 컬럼 생략) |                |          |

EDA

# 기초 통계량 확인

- → 데이터의 중심 경향과 분산 파악
- **평균, 중앙값, 최빈값**으로 중심 경향 파악
- **표준편차, 분산**으로 데이터의 흩어진 정도 확인
- `describe()`로 요약 통계량을 한 번에 확인 가능



```
# 기초 통계 분석
print(' 평균 :\n', df[['age', 'fare']].mean())
print(' 중앙값 :\n',
      df[['age', 'fare']].median())
print(' 최빈값 :\n',
      df[['age', 'fare']].mode().iloc[0])

df.describe()
```



## Outlier

# 이상치를 찾는 세 가지 접근

01

**IQR (사분위수 범위)**

( $Q1 - 1.5 * IQR$ ) ~  
( $Q3 + 1.5 * IQR$ ) 범위를  
벗어나면 이상치로 판단

02

**Z-score**

표준화 후 절댓값이  
3(또는 2)을 넘으면  
이상치로 간주

03

**도메인 지식**

해당 분야의 상식적인 값  
범위를 기준으로 이상치 판단  
ex. 나이 100세 초과

## Outlier & N/A

# 이상치 및 결측치 탐지

- age : 177건 결측 (약 20%) → **대체 필요**
- embarked/embark\_town : 2건 결측 → **소수라 처리 용이**
- deck : 688건 결측 (약 77%) → **컬럼 삭제 고려**
- fare : IQR 기준 116건이 이상치 → **고가 티켓 소수 존재**
- age : 11건이 이상치 → **상대적으로 적음**

```
# 결측값 탐지
print(df.isnull().sum())
```

```
age          177
embarked      2
embark_town   2
deck         688
```

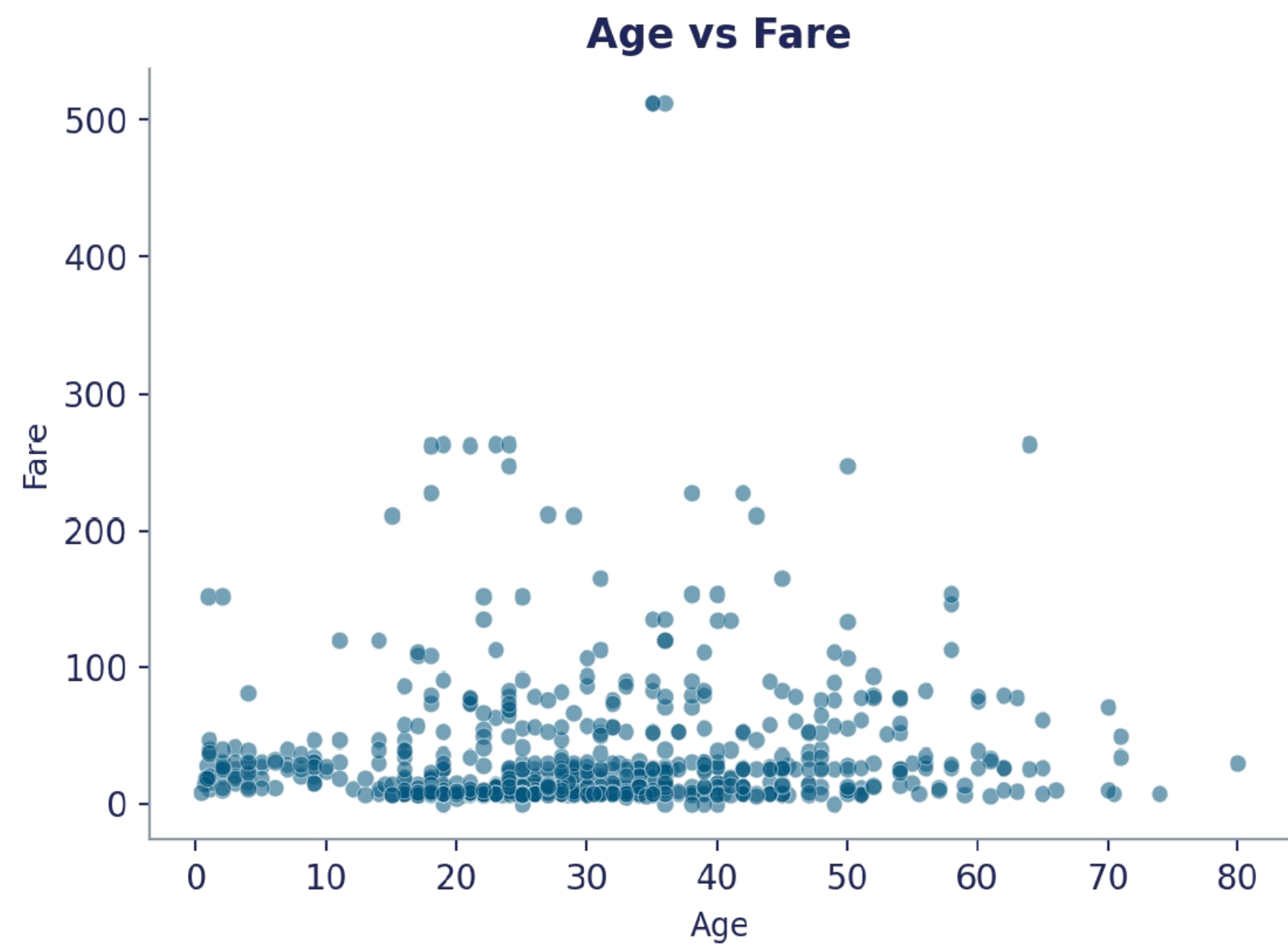
```
# 이상치 탐지 (IQR)
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
outliers = ((df['fare'] < Q1-1.5*IQR) |
             (df['fare'] > Q3+1.5*IQR))
print('fare 이상치 개수:', outliers.sum())
```

fare 이상치 개수 : 116

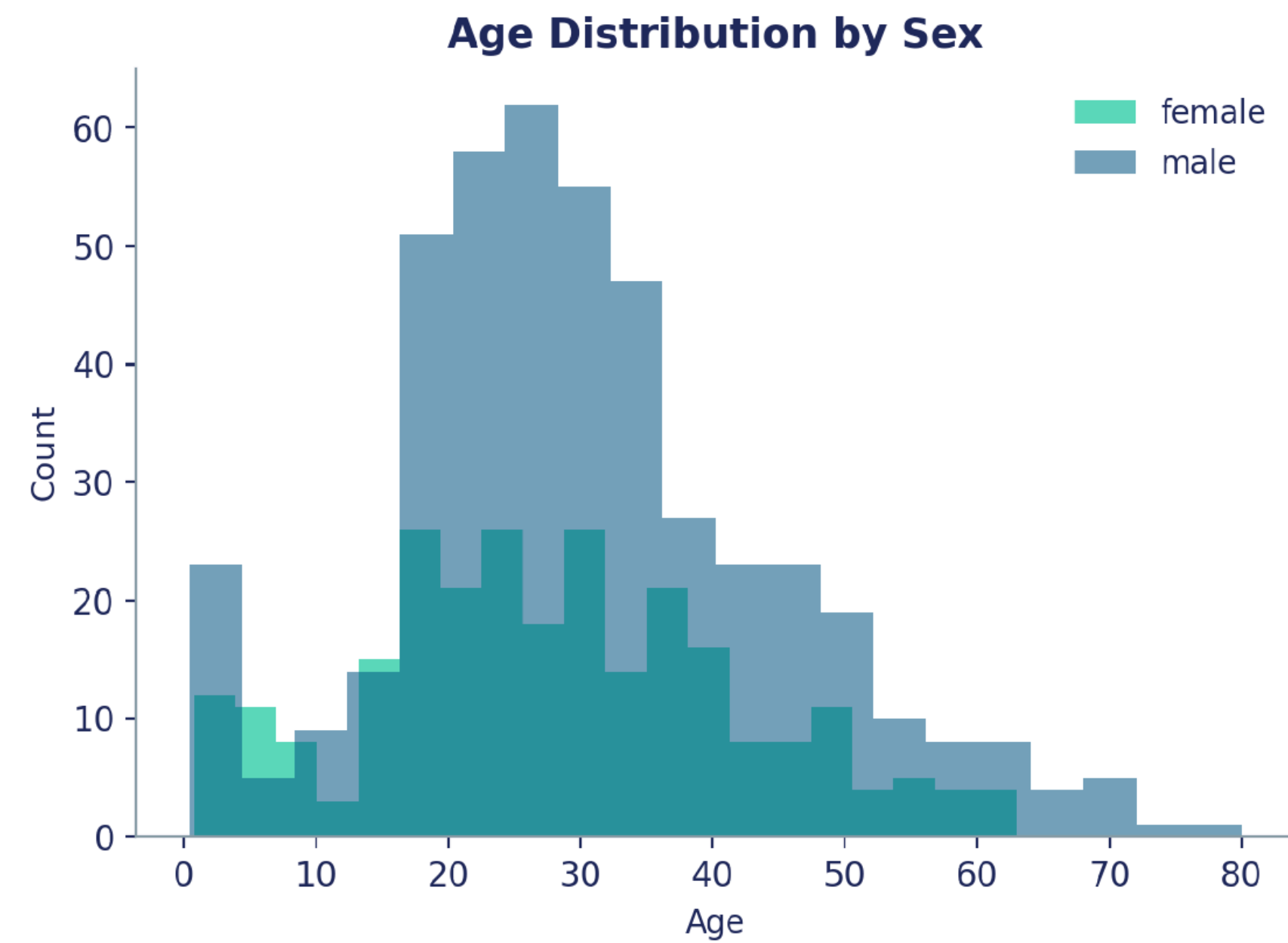
age 이상치 개수 : 11

## Data Visualization

# 산점도 | 히스토그램



`plt.scatter(df['age'], df['fare'])`

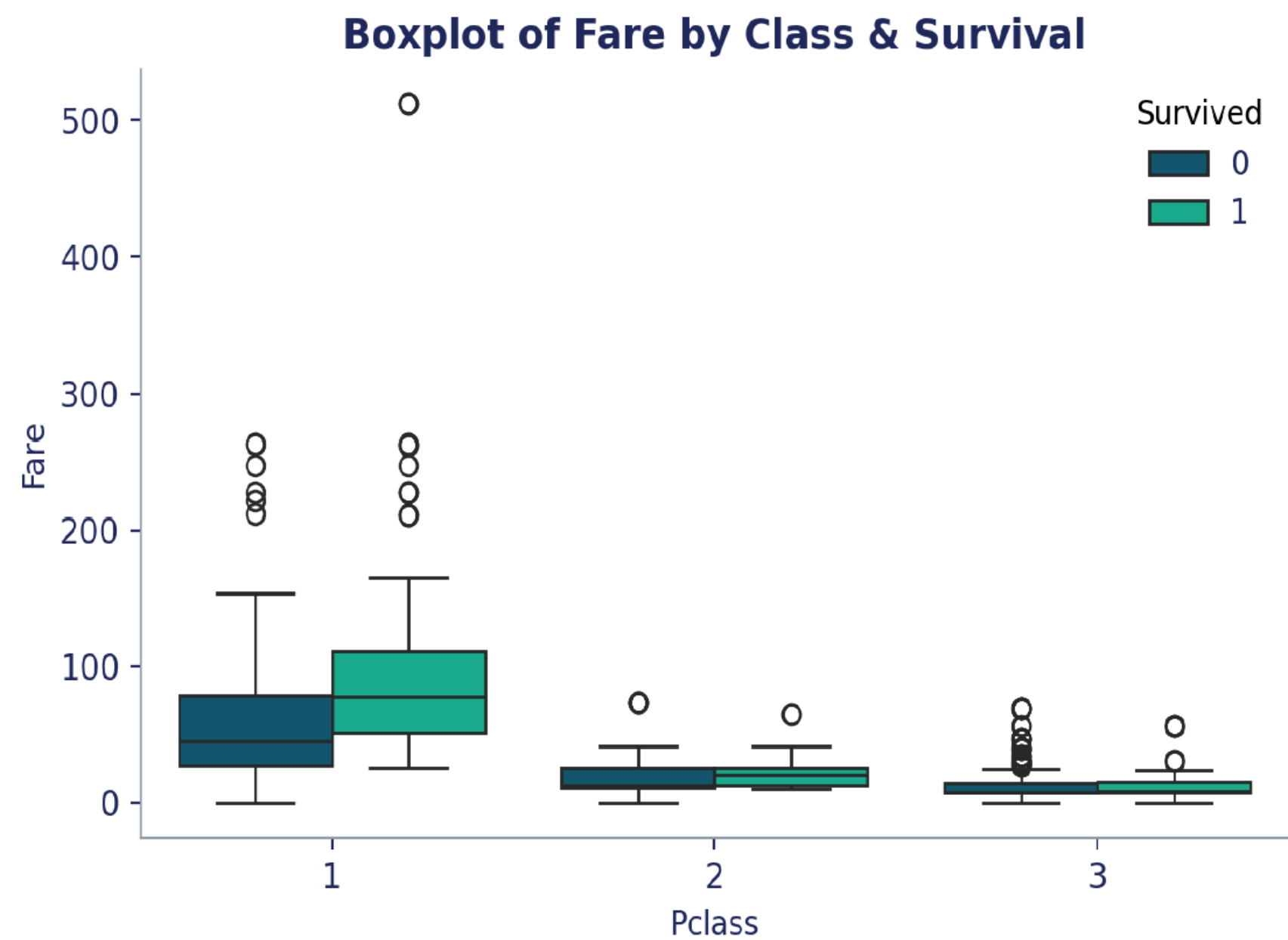


`plt.hist(...)` — 성별에 따른 나이 분포 비교

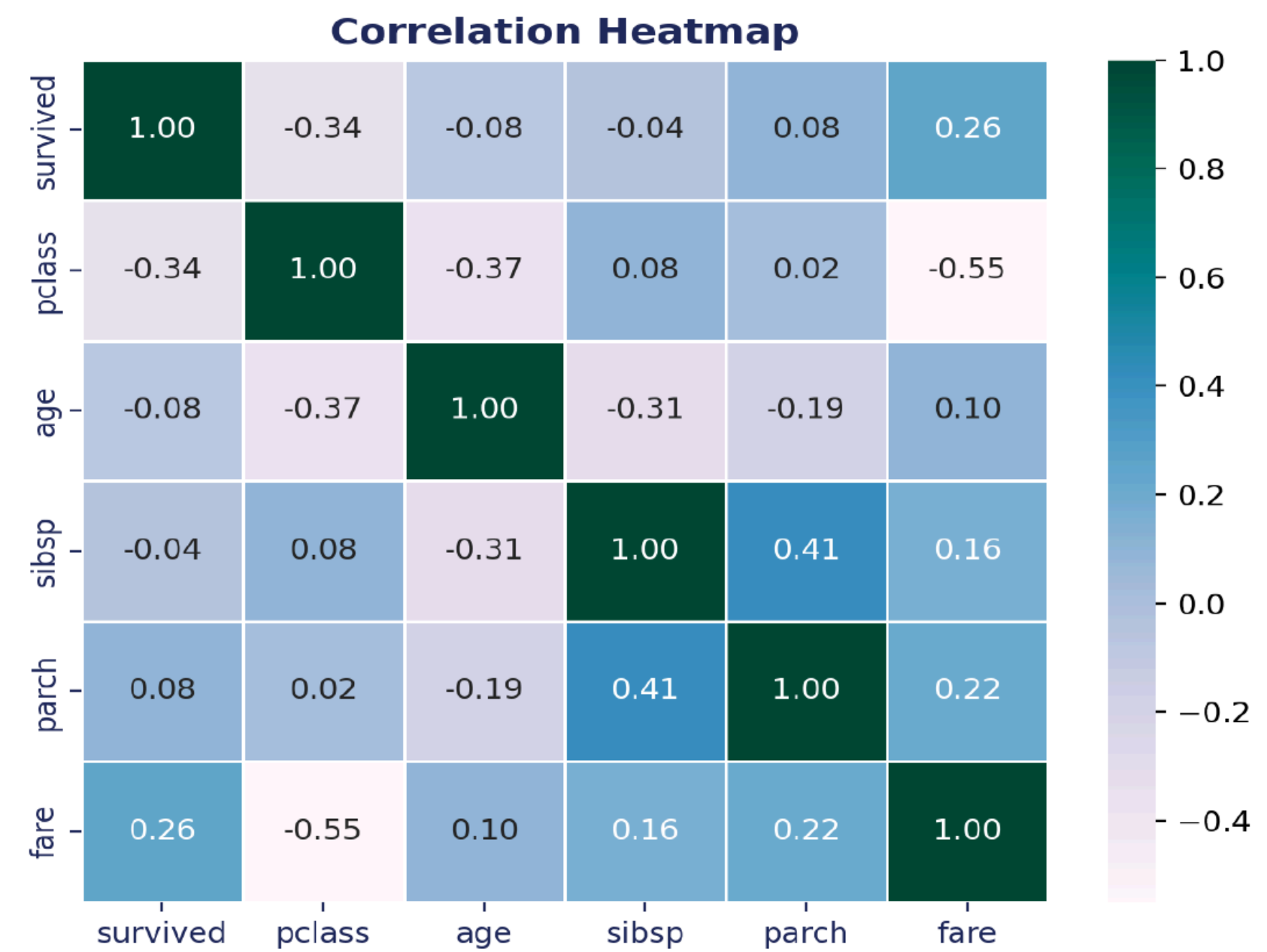


## Data Visualization

# 박스플롯 | 히트맵



`sns.boxplot()` — 등급·생존여부별 운임 분포

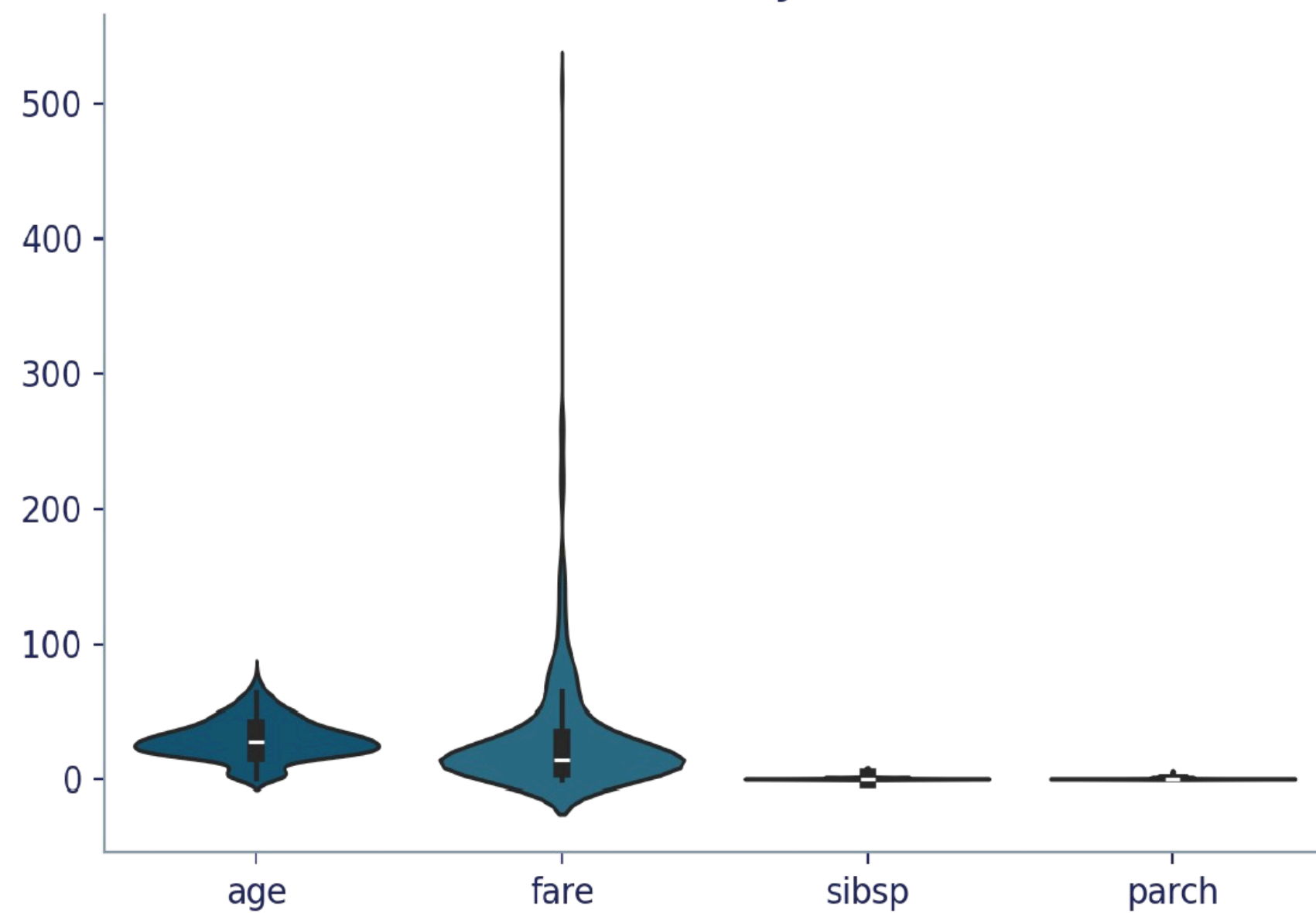


`sns.heatmap()` — 수치형 변수 간 상관관계

## Data Visualization

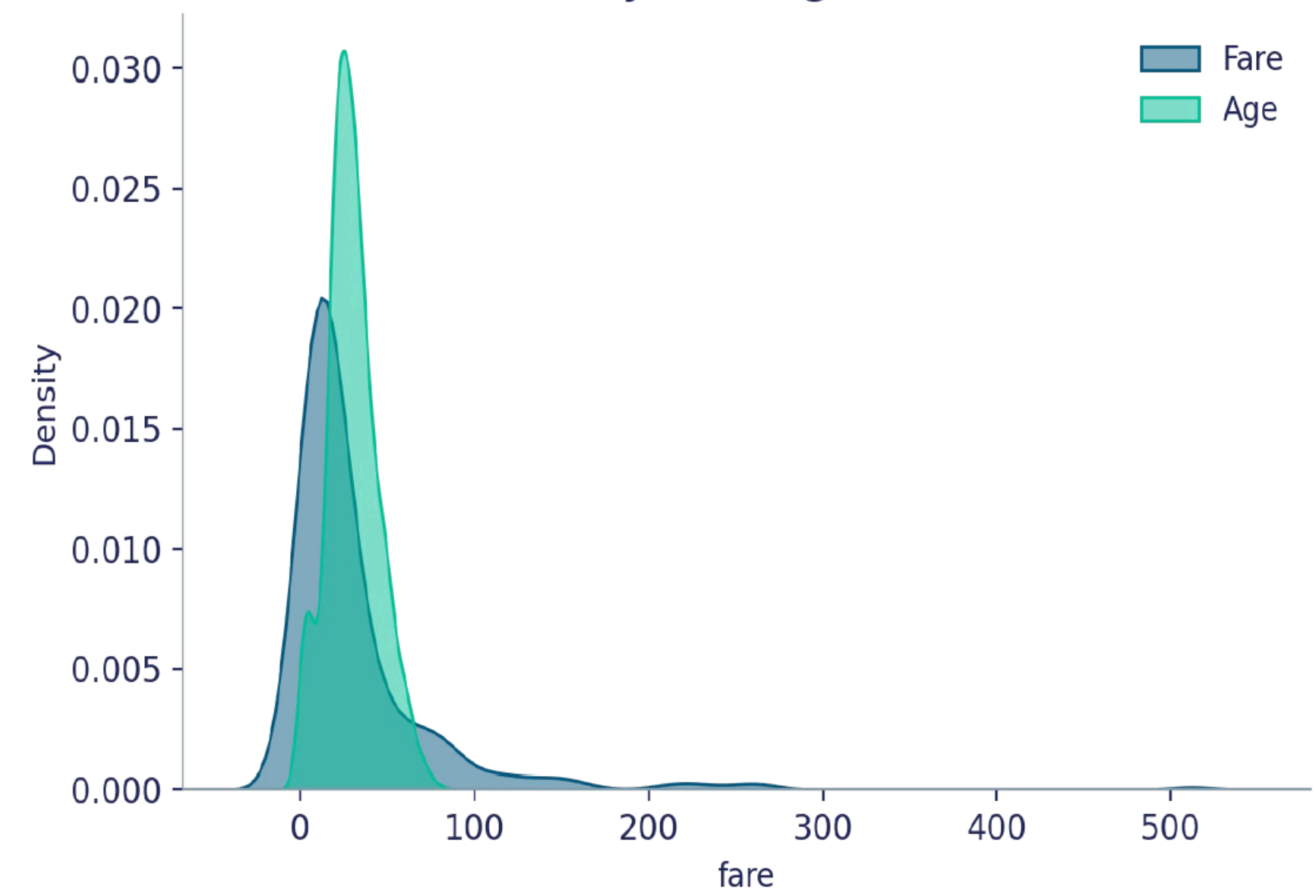
# 바이올린 | 밀도 플롯

Violin Plot of Key Features



`sns.violinplot()` — 분포의 모양까지 함께 확인

Density Plot: Age & Fare

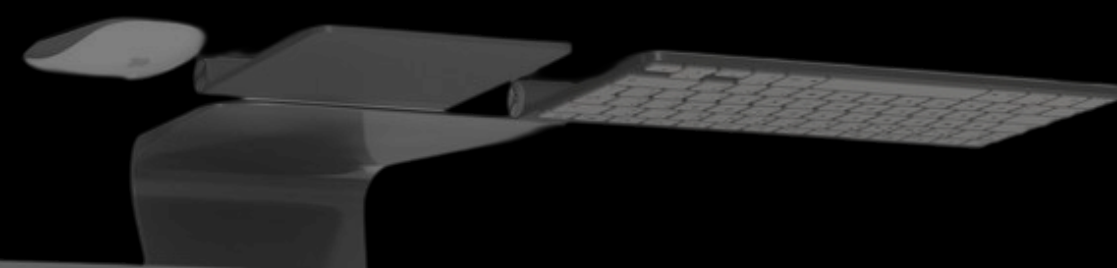


`sns.kdeplot()` — Age·Fare 의 확률밀도 비교

# 02

titanic dataset

엔지니어링  
피쳐





---

# 피처 엔지니어링 :

---

원본 데이터를 가공하여 ( 머신러닝 알고리즘의 예측 성능을 높이는 )  
변수로 변환 및 생성하는 과정.

---

크게 4가지의 주요 기법으로 구성되어 있어요.

- 01 데이터 전처리 : 수집한 원시 데이터를 분석이나 인공지능 모델 학습에 적합한 고품질 형태로 가공하고 정제하는 과정
  - 02 변수 선택 : 모델의 성능을 높이고 과적합을 막기 위해 중요하고 쓸모 있는 입력 변수만 골라내는 과정
  - 03 파생변수 생성 : 기존 데이터 조합으로 새로운 의미를 가진 변수를 만드는 과정
  - 04 스케일링 : 변수마다 다른 값의 범위를 일정한 수준으로 맞추는 과정
-

# 피처엔지니어링 주요 기법

01

## 데이터 전처리

- 결측치 처리
- 이상치 제거
- 데이터 정제
- 노이즈 감소
- 데이터 타입 변환
- 중복 제거

02

## 변수 선택

- 중요 변수 선별
- 차원 축소 적용
- 다중공선성 제거
- PCA 활용
- 상관관계 분석
- 불필요한 피처 제거

03

## 파생변수 생성

- 기존 변수 조합
- 수학적 변환
- 날짜 피처 추출
- 상호작용 변수
- 집계 변수 생성
- 비율 변수 생성

04

## 스케일링

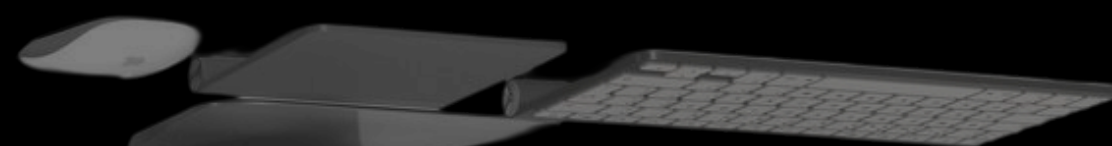
- 정규화 적용
- 표준화 수행
- Min-Max 스케일링
- Robust 스케일링
- 변수 범위 통일
- 모델 성능 개선



# 03

titanic dataset

데이터 전처리



## Feature Engineering

# 결측치 처리

- age : 177건 결측 → **중앙값으로 대체**
- embarked / embark\_town : 2건 결측 → **최빈값으로 대체**
- deck : 688건 결측(77%) → **결측 비율이 너무 높아 컬럼 제거**
- SimpleImputer 클래스로 일괄 처리 가능



```
from sklearn.impute import SimpleImputer

# 중앙값으로 채우는 전략
imputer = SimpleImputer(strategy='median')
df['age'] = imputer.fit_transform(df[['age']])

df['embarked'] = df['embarked']\
    .fillna(df['embarked'].mode()[0])
df = df.drop(columns=['deck'])
```

## Feature Engineering

# 이상치 제거 & 원-핫 인코딩

- → 원-핫 인코딩 포인트
- 각 범주를 새로운 이진 변수로 변환 (sex→sex\_male)
- 순서가 없는 **명목형 변수(sex, embarked)에 적합**
- drop\_first=True로 더미변수 함정(다중공선성) 방지
- 범주가 너무 많으면 컬럼 수가 급증하는 단점



```
# 이상치 제거 (IQR)
df = df[~((df['fare'] < Q1-1.5*IQR) |
          (df['fare'] > Q3+1.5*IQR))]

# 범주형 데이터 처리 - 원 - 핫 인코딩
cat_cols = ['sex', 'embarked']
df = pd.get_dummies(df, columns=cat_cols,
                    drop_first=True)
```



## Feature Engineering

# 레이블 인코딩

- → 레이블 인코딩 포인트
- 각 범주에 고유한 정수를 부여해 하나의 컬럼으로 표현
- **등급처럼 순서가 있는 변수**에 적합
- 순서가 없는 변수에 사용하면 모델이 크기 관계를 잘못 학습할 수 있음
- pclass처럼 이미 숫자로 순서가 표현된 컬럼도 같은 원리



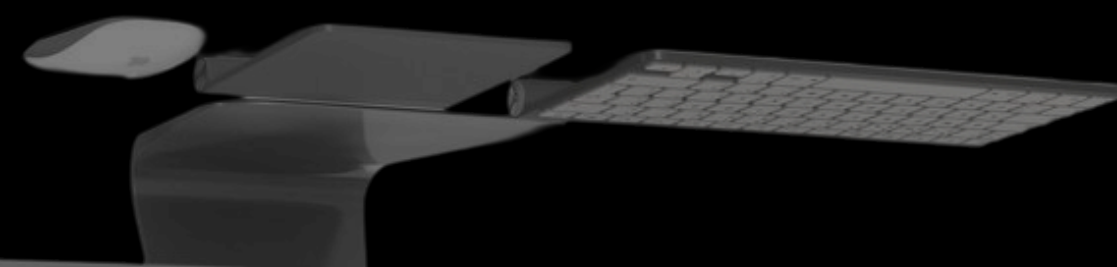
```
# 레이블 인코딩 - 순서가 있는 범주형 변수
# class: Third(3 등급) < Second(2 등급) < First(1
등급)
class_map = {'Third': 0, 'Second': 1, 'First':
2}
df['class_encoded'] =
df['class'].map(class_map)

print(df[['class',
'class_encoded']].drop_duplicates())
```

# 04

titanic dataset

변수선택



---

# 변수선택과 차원축소

---

과적합을 방지하고, 해석 가능성이 향상되며  
( 계산 비용을 절감 ) 할 수 있어 유용.

---

변수선택은 데이터에서 중요한 변수만 골라서 사용하는 기법이에요.

변수선택 방법에는 filter method, wrapper method, embedded method가 있어요.

차원 축소는 기존 변수를 새로운 변수 (저차원 표현)로 변환하는 기법이에요.

차원 축소 방법에는 PCA, t-SNE, LDA, Auto Encoder가 있어요.

---



# 변수 선택 방법

01

## 상관계수

피어슨/스피어만 상관계수로  
연속형 변수 간의 선형적  
관련성을 측정함

02

## 카이제곱 검정

변주형 독립변수와  
범주형 타깃 변수 간의  
독립성을 검정함

03

## ANOVA

범주형 독립변수에 따른  
연속형 타깃 변수의 평균  
차이를 비교함

# 변수 선택 방법

04

## 전진 선택

변수가 없는 상태에서 출발하여  
예측력이 가장 좋은 변수를  
하나씩 추가해나가는 방식

05

## 후진 제거

모든 변수를 포함한 상태에서 시작하여,  
모델 성능에 기여도가 가장 낮은 변수를  
하나씩 제거해 나가는 방식



# 변수 선택 방법

06

## 라쏘 회귀

선형 회귀 손실함수에 L1 가중치 규제(절댓값 합)를 추가하여 중요도가 낮은 변수의 회귀계수를 완벽히 0으로 만들어 자동으로 변수를 선택함

07

## Random Forest

여러 의사결정나무를 학습시킬 때 특정 변수가 불순도를 얼마나 줄여주는지 측정하여 변수 중요도를 계산하고 상위 변수를 선택함

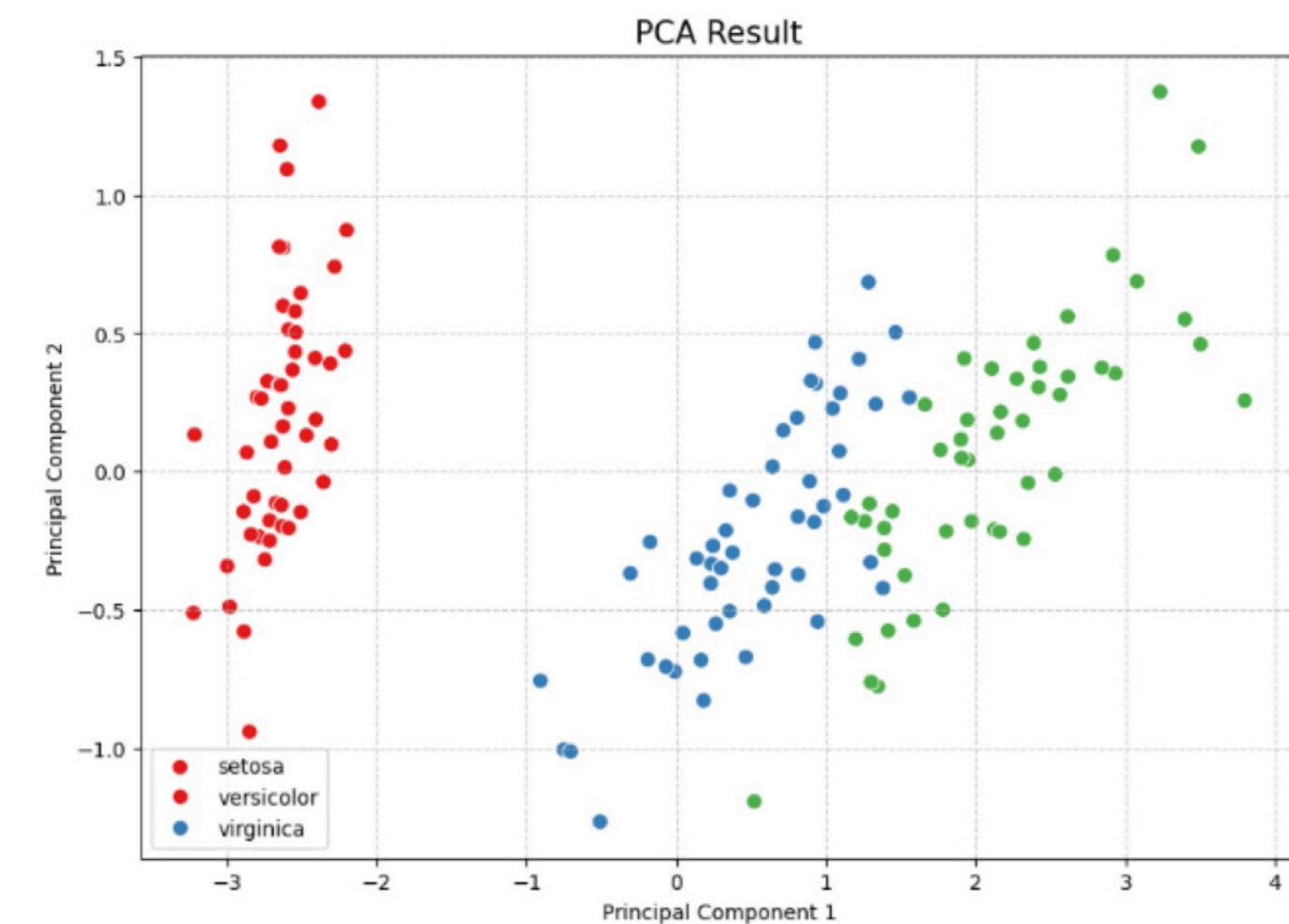
## Dimension Reduction

# 차원 축소 방법

01

## PCA

고차원의 데이터를 저차원으로 변환하여  
데이터의 분산을 최소화



```
from sklearn.decomposition import PCA  
pca = PCA(n_components=2).fit_transform(X) # X는 입력 데이터, 2차원으로 축소
```

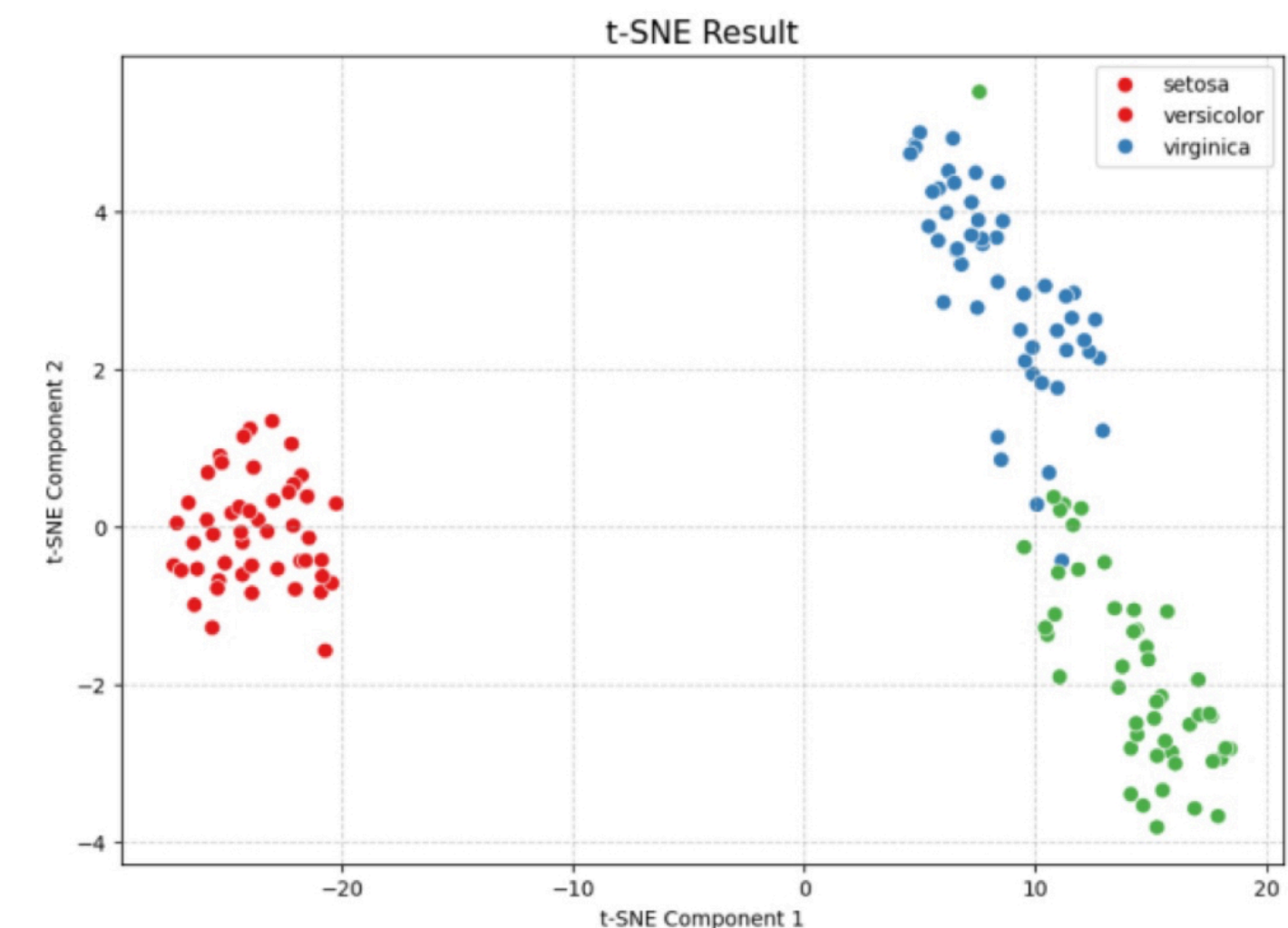
## Dimension Reduction

# 차원 축소 방법

02

## t-SNE

비선형 차원 축소로 데이터간의  
유사성을 보존하여 저차원으로 변환



```
from sklearn.manifold import TSNE  
tsne = TSNE(n_components=2).fit_transform(X) # X는 입력 데이터, 2차원으로 축소
```



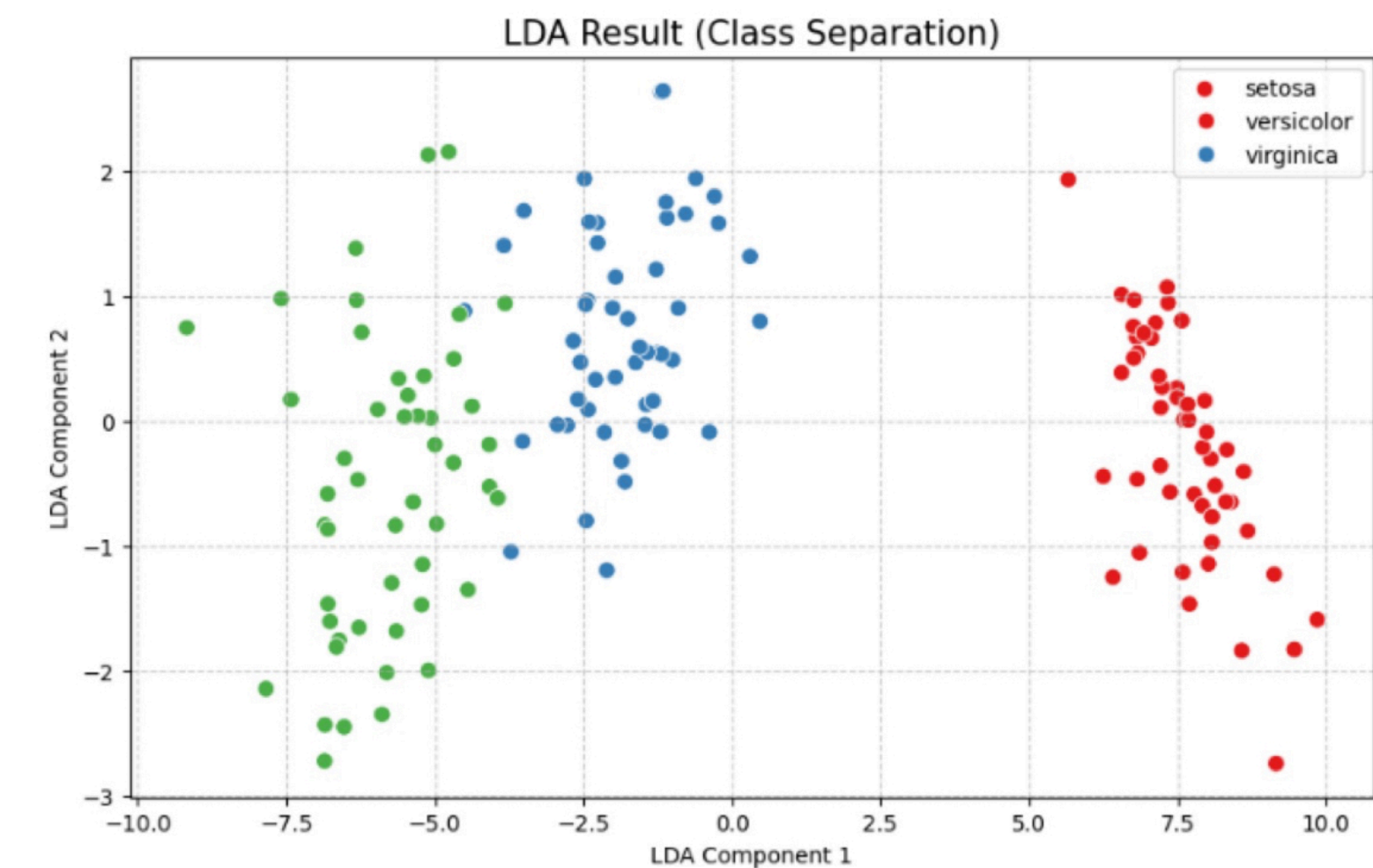
## Dimension Reduction

# 차원 축소 방법

03

## LDA

선형판별 분석법으로 주로 분류 문제에서 사용



```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis  
  
lda = LinearDiscriminantAnalysis(n_components=2).fit_transform(X, y) # X는 입력 데이터, y는 클래스 레이블
```

# 차원 축소 방법

04

## Auto Encoder

신경망 기반 차원 축소 기법

```
from keras.models import Model
from keras.layers import Input, Dense

input_data = Input(shape=(X.shape[1],)) # 입력 데이터의 차원
encoded = Dense(2, activation='relu')(input_data) # 2차원으로 인코딩
autoencoder = Model(input_data, encoded)
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(X, X, epochs=50, batch_size=16) # 입력 데이터를 재구성하는 방식으로 훈련
```

# 05

titanic dataset

파생변수생성





---

# 파생 변수 Derived Variable :

---

기존의 변수로부터 생성된 새로운 변수로,  
( 데이터에 추가적인 정보를 제공 ) 하고 모델의 학습효과를 향상시킴.

---

크게 4가지의 방법이 있어요.

- 01 수치적 변형 : 기존 수치형 변수에 스케일링, 로그 변환, 상호작용 계산 등의 수학적 연산을 적용
  - 02 범주형 변수 변환 : 텍스트/기호 형태의 범주형 데이터를 수치형 데이터로 변환, 의미 있는 그룹으로 재범주화
  - 03 날짜 / 시간 변형 : 시간 정보에서 주기성이나 시간적 의미를 지닌 새로운 특성을 추출
  - 04 텍스트 데이터 변형 : 비정형 텍스트에서 단어 빈도 등을 계산하거나 임베딩을 통해 모델이 이해할 수 있는 수치적 특성으로 추출
-

Derived Variable

# 파생변수 생성

- **titanic 데이터셋** 기준 생성한 파생변수
  - 가족 규모와 동승 여부
  - 1인당 운임
  - 연령대 구간화



```
# 가족 규모 & 동승 여부
df['family_size'] = df['sibsp'] + df['parch'] + 1
df['is_alone'] = (df['family_size'] == 1).astype(int)

# 1인당 운임
df['fare_per_person'] = df['fare'] / df['family_size']

# 연령대 구간화
df['age_group'] = pd.cut(
    df['age'], bins=[0,12,19,35,60,100],
    labels=['child','teen','young_adult',
            'adult','senior'])
```



Derived Variable

## 파생변수 생성의 예시

01

금융 데이터

자산 = 현금 + 주식 + 채권

02

마케팅 데이터

고객 충성도 점수  
= 구매빈도 \* 평균구매액

03

건강 데이터

BMI = 체중 / (신장^2)

## 파생변수 생성 시 고려할 점

01

### 과적합

너무 많은 파생변수를  
생성할 경우, 모델  
훈련 과정에서 과적합 발생

02

### 의미 있는 데이터

생성한 변수가 유의미하고,  
해석 가능해야 함

03

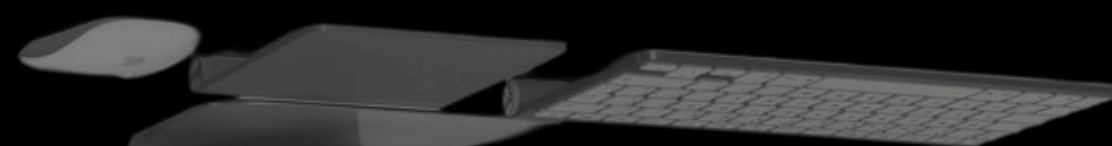
### 변수의 상관관계

기존 변수와 강한  
상관관계를 가진다면  
변수를 선택해야 함

# 06

titanic dataset

## 스케일링





---

# 스케일링 Data Scaling :

---

변수의 범위를 조정하여 모델의 학습 효율성을 높이고,  
( 성능을 향상 ) 시킴

---

모델 수렴 속도가 향상하고 거리 기반 모델의 성능이 향상되며 데이터의 해석 용이성 측면에서 스케일링이 유용해요.

스케일링 기법은 크게 3가지로 구분돼요.

01 정규화 : 각 피쳐 값을 0과 1 사이로 조정

02 표준화 : 각 피쳐의 평균을 0, 표준편차를 1로 조정

03 로버스트 스케일 : 중앙값과 사분위수를 사용하여 스케일링

---

## Data Scaling

# 스케일링 기법 3가지

- **표준화** : 평균 0, 표준편차 1로 조정
  - PCA, 로지스틱 회귀 등에 기본
- **정규화 (Min-Max)** : 값을 0과 1 사이로 조정
- **로버스트 스케일** : 중앙값, 사분위수 사용, 이상치에 강건



```
# 표준화 ( 평균 0, 분산 1 )
standard_scaler = StandardScaler()
df_standardized = pd.DataFrame(
    standard_scaler.fit_transform(df),
    columns=df.columns)
```

```
# 정규화 (0~1 범위 )
min_max_scaler = MinMaxScaler()
df_normalized = pd.DataFrame(
    min_max_scaler.fit_transform(df),
    columns=df.columns)
```

titanic dataset

# 쿠글 W1 세미나

14기 송채영

→ 과제는 깃허브를 통해 공유드린  
파이썬 파일을 열어 빈칸을 채워주세요

2026. 09. 08

감사합니다

